

Introduction to Scientific Computing

ENGR1050 – Lecture 25

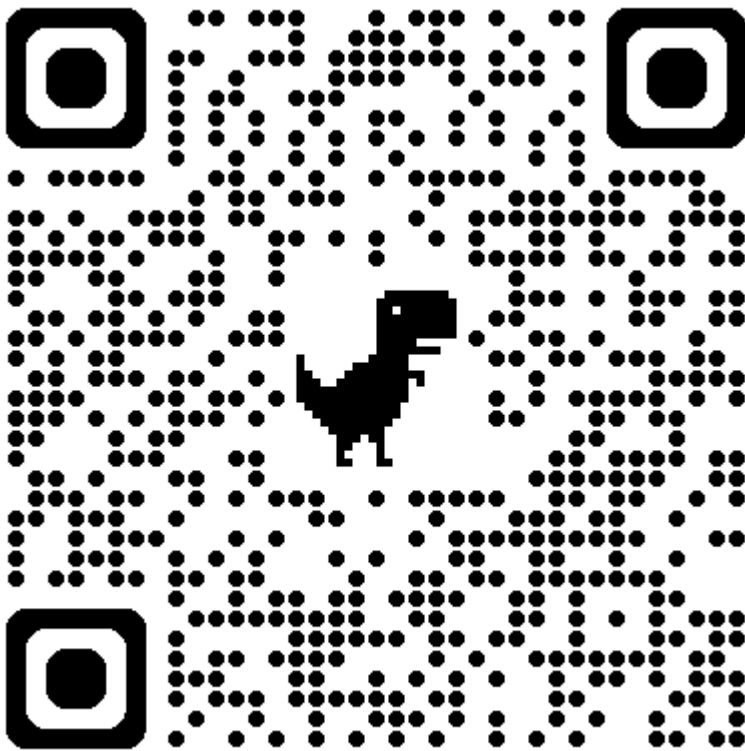
Fall 2025

Nat Trask

University of Pennsylvania

Binary classification

Predictive maintenance





Review

Backprop
Gradient descent
Polynomial fitting
Deep neural networks
Stochastic gradient descent

Classification

Modify a neural network to
predict probability of a true/false
relationship
Serious engineering: predictive
maintenance

Review: Supervised learning

Goal: Given data points (x_i, y_i) , find parameters θ that best fit a model

The Problem

- Model: $\hat{y} = f(x; \theta)$ (prediction)
- Data: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ (observations)
- Loss function: $L(\theta) = \sum_{i=1}^n (y_i - f(x_i; \theta))^2$ (error)

Optimization Approach:

- 1 Minimize loss: find θ^* where $\frac{\partial L}{\partial \theta} = 0$
- 2 **This is root finding!** Apply Newton's method
- 3 Or use optimization form: $\theta_{n+1} = \theta_n - \frac{\partial L / \partial \theta}{\partial^2 L / \partial \theta^2}$

Connection to Engineering:

- Fit helicopter control model to flight data
- Calibrate COVID model (β, γ) to case counts
- Train neural networks (gradient descent is simplified Newton)

Review: gradient descent

Recall the update for Newton's method:

$$x_{n+1} = x_n - \underbrace{\frac{1}{f''(x_n)}}_{\text{step distance}} \underbrace{f'(x_n)}_{\text{step direction}}$$

Computing second derivatives is expensive; if we have n variables, it costs n^2 to compute the gradient of every gradient.

To make this efficient, gradient descent introduces a fixed step size α :

$$x_{n+1} = x_n - \alpha f'(x_n) \leftarrow \text{Compute with automatic differentiation!}$$

Step size. α must be tuned to ensure convergence. Too large can overshoot; too small leads to slow progress.

Review: computing gradients with backprop

```
import torch
```

```
x = torch.tensor(3.0, requires_grad=True)
```

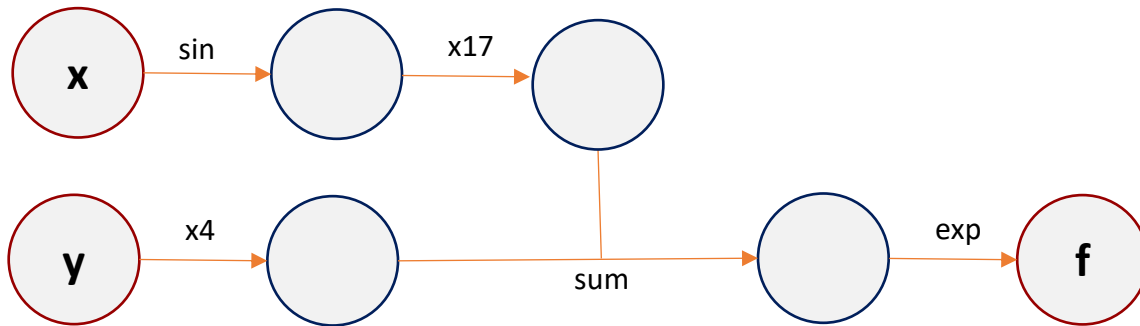
```
y = torch.tensor(4.0, requires_grad=True)
```

```
f = torch.exp(17 * torch.sin(x) + 4*y)
```



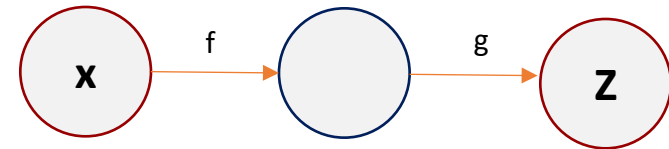
0.0s

Python



Step 1. Torch decomposes functions into a *computational graph*

$$z = f(g(x))$$



$$\frac{dz}{dx} = f'(g(x)) \cdot g'(x)$$

Step 2. Traversing the graph backward with the chain rule gives gradients

Review: Four step cookbook to take a gradient

Step 1: Define variables, set requires_grad=True

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True) # A vector  
y = torch.tensor(3, requires_grad=True) # A scalar
```

Step 2: Evaluate forward pass of function to differentiate

```
f = x**2 + y * x # Some function of x and y
```

Step 3: Evaluate backward() on output to compute gradients

```
f.backward() # Compute gradients
```

Step 4: Access dfdx and dfdy gradients via .grad attribute

```
print(x.grad) # Gradient of f w.r.t. x  
print(y.grad) # Gradient of f w.r.t. y
```

Warning: Need to run `x.grad.zero_()` before next `backward()`

How did we do this for polynomial fitting? Forward pass

```
class modelClass:
    def __init__(self, a0, b0):
        # Initialize a linear model  $y = a*x + b$ 
        # a0 - (float) initial value for slope
        # b0 - (float) initial value for intercept

        self.a = torch.tensor(a0, requires_grad=True)
        self.b = torch.tensor(b0, requires_grad=True)

    def forward(self, x):
        # Given input x (assumed to be a scalar shape torch tensor), return predicted  $y = a*x+b$ 
        return self.a * x + self.b
```

How did we do this for polynomial fitting? Backward pass

```
# Initialize a linear model with a = b = 0
model = modelClass(a0=0.0, b0=0.0)

# Set up loop to train model with gradient descent
learning_rate = 0.0001
num_steps = 200
for step in range(num_steps):

    # Step 1: Take forward pass by evaluating model on each data point and computing the total loss
    total_loss = 0.0
    for i in range(len(x_data)):
        x = x_data[i]
        y_true = y_data[i]

        # Forward pass: compute predicted y
        y_pred = model.forward(x)

        # Compute loss (mean squared error)
        loss = (y_pred - y_true)**2
        total_loss += loss.item()

    # Step 2: Backward pass: compute gradients using automatic differentiation
    loss.backward()

    # Step 3: Update parameters using gradient descent
    with torch.no_grad():
        model.a -= learning_rate * model.a.grad
        model.b -= learning_rate * model.b.grad

    # Zero gradients after updating - otherwise they will accumulate
    model.a.grad.zero_()
    model.b.grad.zero_()
```

Step 1. Use model to evaluate loss

Step 2. Take gradient of loss with respect to model parameters

Step 3. Use gradient to update parameters

From polynomials to deep neural networks

$$y = W_1 + W_2x$$

```
class modelClass:
    def __init__(self, a0, b0):
        # Initialize a linear model y = a*x + b
        # a0 - (float) initial value for slope
        # b0 - (float) initial value for intercept

        self.a = torch.tensor(a0, requires_grad=True)
        self.b = torch.tensor(b0, requires_grad=True)

    def forward(self, x):
        # Given input x (assumed to be a scalar shape torch tensor), return predicted y = a*x+b
        return self.a * x + self.b
```

$$\begin{aligned} \mathbf{h}_1 &= \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \\ \mathbf{h}_2 &= \sigma(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2) \\ &\vdots \\ \mathbf{h}_N &= \sigma(\mathbf{W}_N \mathbf{h}_{N-1} + \mathbf{b}_N) \\ \mathbf{y} &= \mathbf{W}_{out} \mathbf{h}_N \end{aligned}$$

```
class MLP_modelClass:
    def __init__(self, input_dim, hidden_dim, output_dim):
        # Initialize a 2-layer MLP
        # input_dim - dimension of input (e.g., 1 for scalar input)
        # hidden_dim - number of neurons in hidden layer
        # output_dim - dimension of output (e.g., 1 for scalar output)

        # Layer 1: input -> hidden
        self.W1 = torch.randn(hidden_dim, input_dim, requires_grad=True)
        self.b1 = torch.randn(hidden_dim, requires_grad=True)

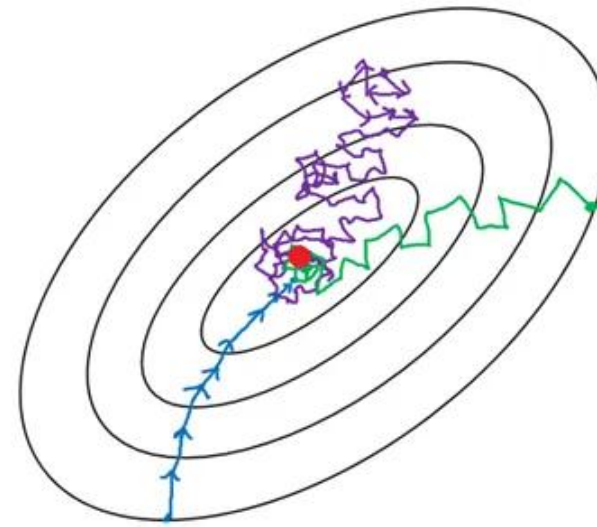
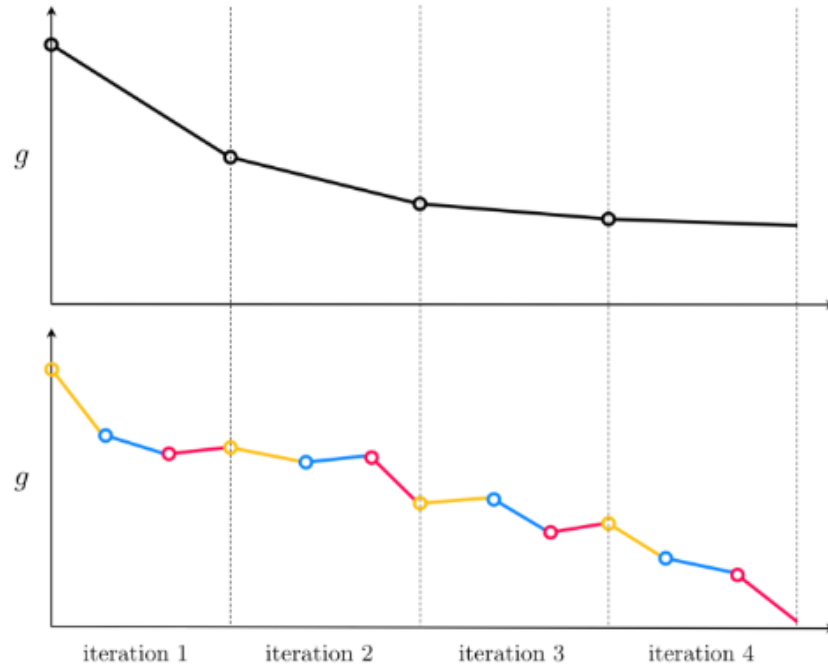
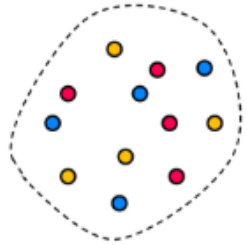
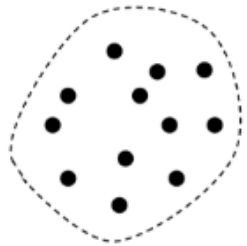
        # Layer 2: hidden -> output
        self.W2 = torch.randn(output_dim, hidden_dim, requires_grad=True)
        self.b2 = torch.randn(output_dim, requires_grad=True)

    def forward(self, x):
        # Layer 1: compute h1 = tanh(W1*x + b1)
        z1 = self.W1 @ x + self.b1    # Linear transformation
        h1 = torch.tanh(z1)           # Activation function

        # Layer 2: compute y = W2*h1 + b2
        y = self.W2 @ h1 + self.b2    # Linear transformation (no activation on output)

        return y
```

Introducing minibatching (AKA *stochastic gradient descent*)



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

At each step of training:

- Randomly select $N_{batchsize}$ datapoints from your dataset
- Perform a gradient descent update as usual



Review

Backprop
Gradient descent
Polynomial fitting
Deep neural networks
Stochastic gradient descent

Classification

**Modify a neural network to
predict probability of a true/false
relationship**

Binary classification AKA the cats and dogs problem



or



Input image

Convolution Layer

ReLU Layer

Pooling Layer

Fully Connected Layer

Output Classes

Dog

Not Dog

In engineering

- Probability of failure
- Probability of defect
- Probability of extreme event

(higher consequences than cats and dogs!)

Today's changes to the code: Two ingredients

Modified model

Change MLP to output a probability instead of an arbitrary number

Modified loss

A new function to minimize that makes sense for probabilities

The Bernoulli distribution (AKA coin flipping)

A **binary probability distribution** (also called a **Bernoulli distribution**) is the simplest type of probability distribution that models outcomes with exactly two possible values.

Definition

A random variable X follows a binary distribution if:

$$P(X = 1) = p$$

$$P(X = 0) = 1 - p$$

where $p \in [0, 1]$ is the **probability parameter**.

Key Properties

Property	Formula	Description
Outcomes	$X \in \{0, 1\}$	Only two possible values
Parameter	$p = P(X = 1)$	Probability of "success" (outcome = 1)
Mean	$\mathbb{E}[X] = p$	Expected value
Variance	$\text{Var}(X) = p(1 - p)$	Spread of the distribution

Common Notation

The probability mass function (PMF) can be written compactly as:

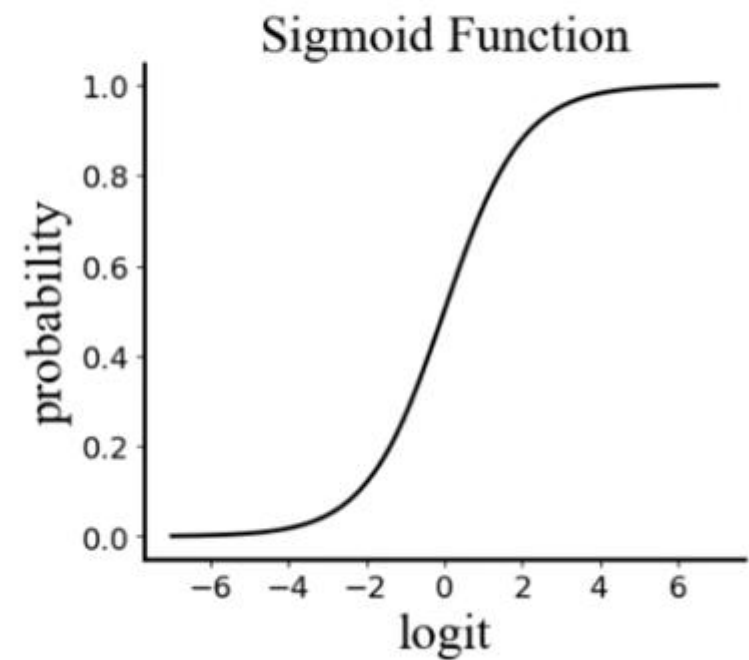
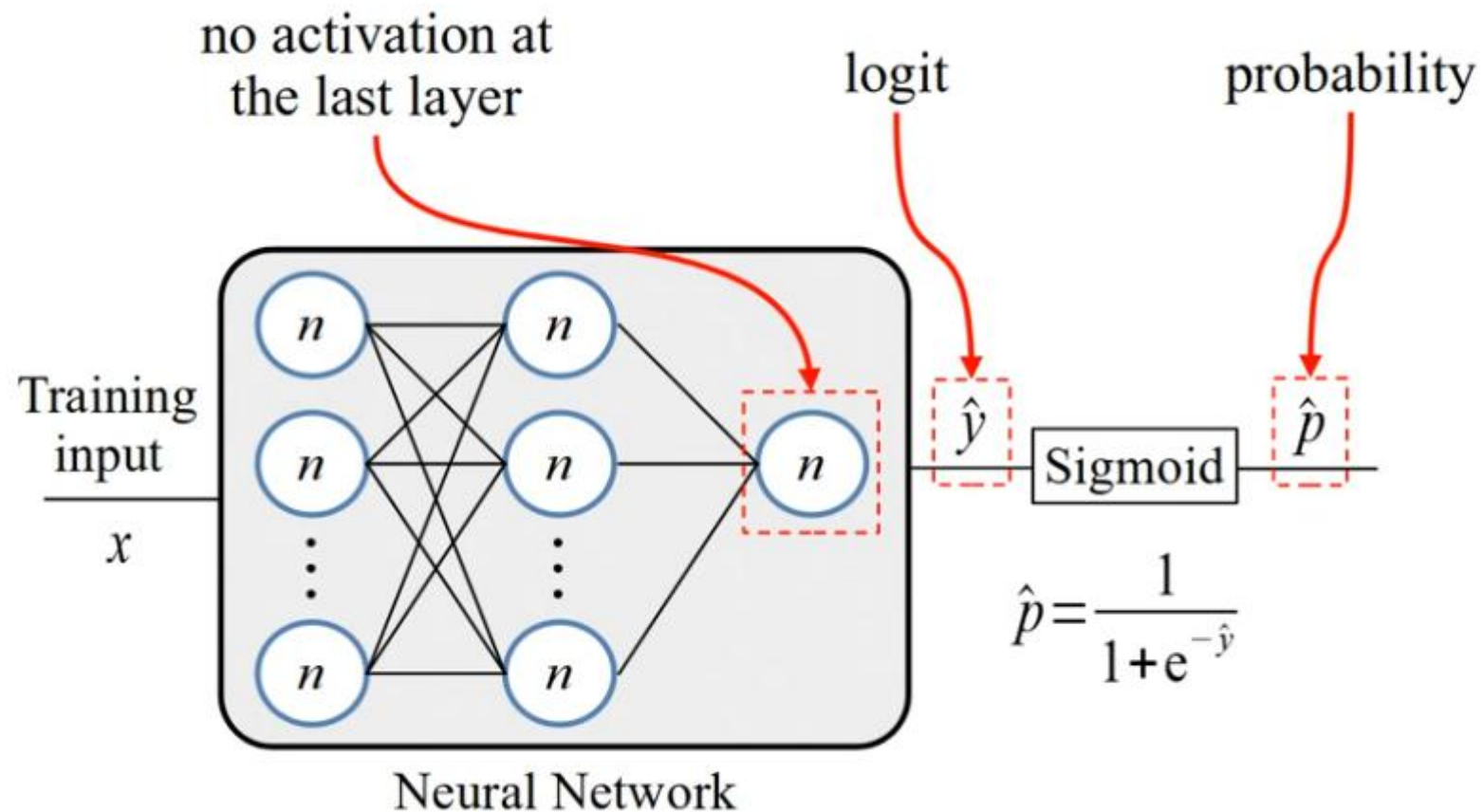
$$P(X = x) = p^x(1 - p)^{1-x}, \quad x \in \{0, 1\}$$

This single equation covers both cases:

- When $x = 1$: $P(X = 1) = p^1(1 - p)^0 = p$
- When $x = 0$: $P(X = 0) = p^0(1 - p)^1 = 1 - p$



How can we make a neural network map from black box inputs to a Bernoulli distribution?



[A nice youtube explanation](#)

How can we make a neural network map from black box inputs to a Bernoulli distribution?

```
class MLP_modelClass:
    def __init__(self, input_dim, hidden_dim, output_dim):
        # Initialize a 2-layer MLP
        # input_dim - dimension of input (e.g., 1 for scalar input)
        # hidden_dim - number of neurons in hidden layer
        # output_dim - dimension of output (e.g., 1 for scalar output)

        # Layer 1: input -> hidden
        self.W1 = torch.randn(hidden_dim, input_dim, requires_grad=True)
        self.b1 = torch.randn(hidden_dim, requires_grad=True)

        # Layer 2: hidden -> output
        self.W2 = torch.randn(output_dim, hidden_dim, requires_grad=True)
        self.b2 = torch.randn(output_dim, requires_grad=True)

    def forward(self, x):
        # Layer 1: compute h1 = tanh(W1*x + b1)
        z1 = self.W1 @ x + self.b1    # Linear transformation
        h1 = torch.tanh(z1)           # Activation function

        # Layer 2: compute y = W2*h1 + b2
        y = self.W2 @ h1 + self.b2    # Linear transformation (no activation on output)

        return y
```

One line change!
At output of MLP, add a
sigmoid to force output in [0,1]



```
class BinaryClassificationModel:
    """Simple logistic regression model for binary classification."""
    def __init__(self, input_dim, hidden_dim, output_dim):
        # Initialize a 2-layer MLP
        # input_dim - dimension of input (e.g., 1 for scalar input)
        # hidden_dim - number of neurons in hidden layer
        # output_dim - dimension of output (e.g., 1 for scalar output)

        # Layer 1: input -> hidden
        self.W1 = torch.randn(hidden_dim, input_dim, requires_grad=True)
        self.b1 = torch.randn(hidden_dim, requires_grad=True)

        # Layer 2: hidden -> output
        self.W2 = torch.randn(output_dim, hidden_dim, requires_grad=True)
        self.b2 = torch.randn(output_dim, requires_grad=True)

    def forward(self, x):
        # Layer 1: compute h1 = tanh(W1*x + b1)
        z1 = self.W1 @ x + self.b1    # Linear transformation
        h1 = torch.tanh(z1)           # Activation function

        # Layer 2: compute y = W2*h1 + b2
        y = self.W2 @ h1 + self.b2    # Linear transformation (no activation on output)

        # NEW: Apply sigmoid to output to get probability
        output_probability = torch.sigmoid(y)

        return output_probability
```

A new loss function for probabilities: the binary cross-entropy loss

For classification, we can't use MSE loss. Instead, we use **binary cross-entropy** loss:

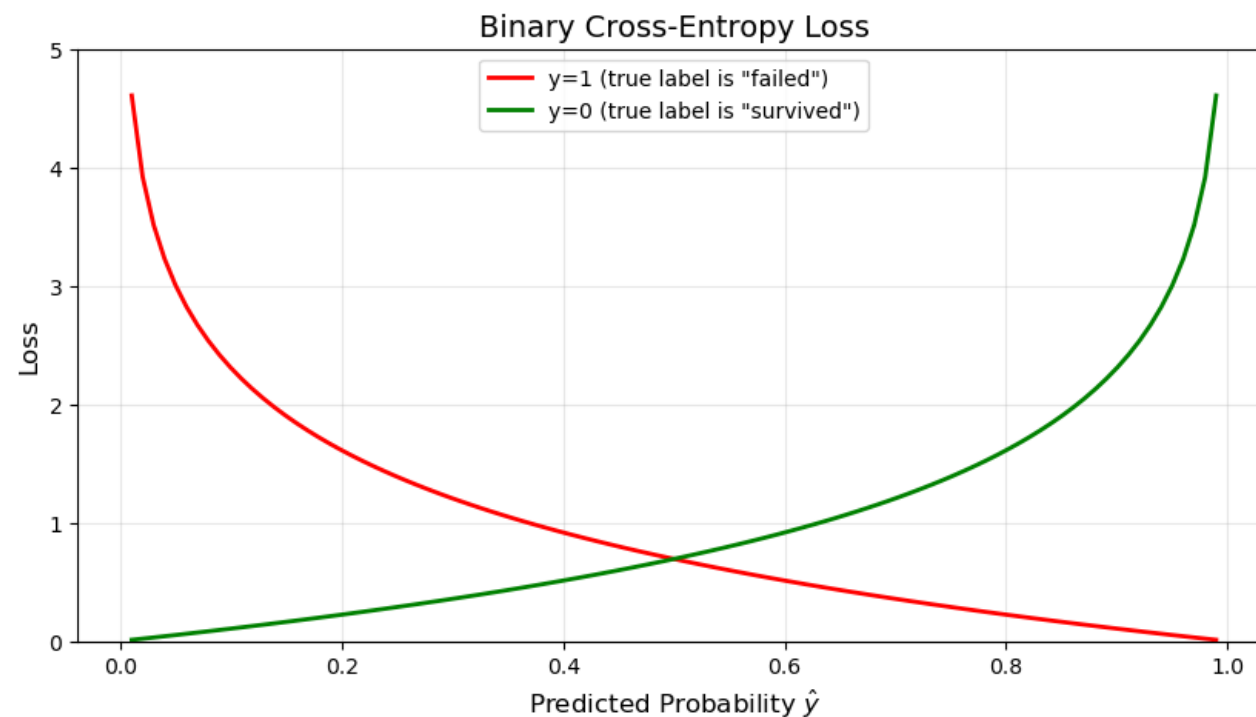
$$\text{BCE}(y, \hat{y}) = - \underbrace{[y \log(\hat{y})]}_{\text{red}} + \underbrace{(1 - y) \log(1 - \hat{y})}_{\text{green}}$$

where:

- $y \in \{0, 1\}$ is the true label
- $\hat{y} \in (0, 1)$ is the predicted probability

Intuition:

- If $y = 1$ (failed), we want $\hat{y} \approx 1$, so $-\log(\hat{y})$ is small
- If $y = 0$ (survived), we want $\hat{y} \approx 0$, so $-\log(1 - \hat{y})$ is small
- Wrong predictions are heavily penalized (logarithm goes to infinity)



First term.

Infinite penalty if model predicts
no event when it should

Second term.

Infinite penalty if model predicts
event when it shouldn't

The old loss from Monday

```
# Initialize a linear model with a = b = 0
model = modelClass(a0=0.0, b0=0.0)

# Set up loop to train model with gradient descent
learning_rate = 0.0001
num_steps = 200
for step in range(num_steps):

    # Step 1: Take forward pass by evaluating model on each data point and computing the total loss
    total_loss = 0.0
    for i in range(len(x_data)):
        x = x_data[i]
        y_true = y_data[i]

        # Forward pass: compute predicted y
        y_pred = model.forward(x)

        # Compute loss (mean squared error)
        loss = (y_pred - y_true)**2
        total_loss += loss.item()

    # Step 2: Backward pass: compute gradients using automatic differentiation
    loss.backward()

    # Step 3: Update parameters using gradient descent
    with torch.no_grad():
        model.a -= learning_rate * model.a.grad
        model.b -= learning_rate * model.b.grad

    # Zero gradients after updating - otherwise they will accumulate
    model.a.grad.zero_()
    model.b.grad.zero_()
```

Step 1. Use model to evaluate loss

Step 2. Take gradient of loss with respect to model parameters

Step 3. Use gradient to update parameters

The binary cross entropy loss for today

```
def binary_cross_entropy(y_pred, y_true):  
    """  
    Compute binary cross-entropy loss.  
    Args:  
        y_pred: Predicted probabilities (batch_size, 1)  
        y_true: True labels 0 or 1 (batch_size, 1)  
    """  
  
    epsilon = 1e-7 # Small constant to avoid log(0)  
    y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon) # Makes sure predictions are  
    within (0+eps,1-eps) so log doesn't explode  
    loss = -(y_true * torch.log(y_pred) + (1 - y_true) * torch.log(1 - y_pred))  
    return loss
```

New function. Define cross entropy

$$\text{BCE}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

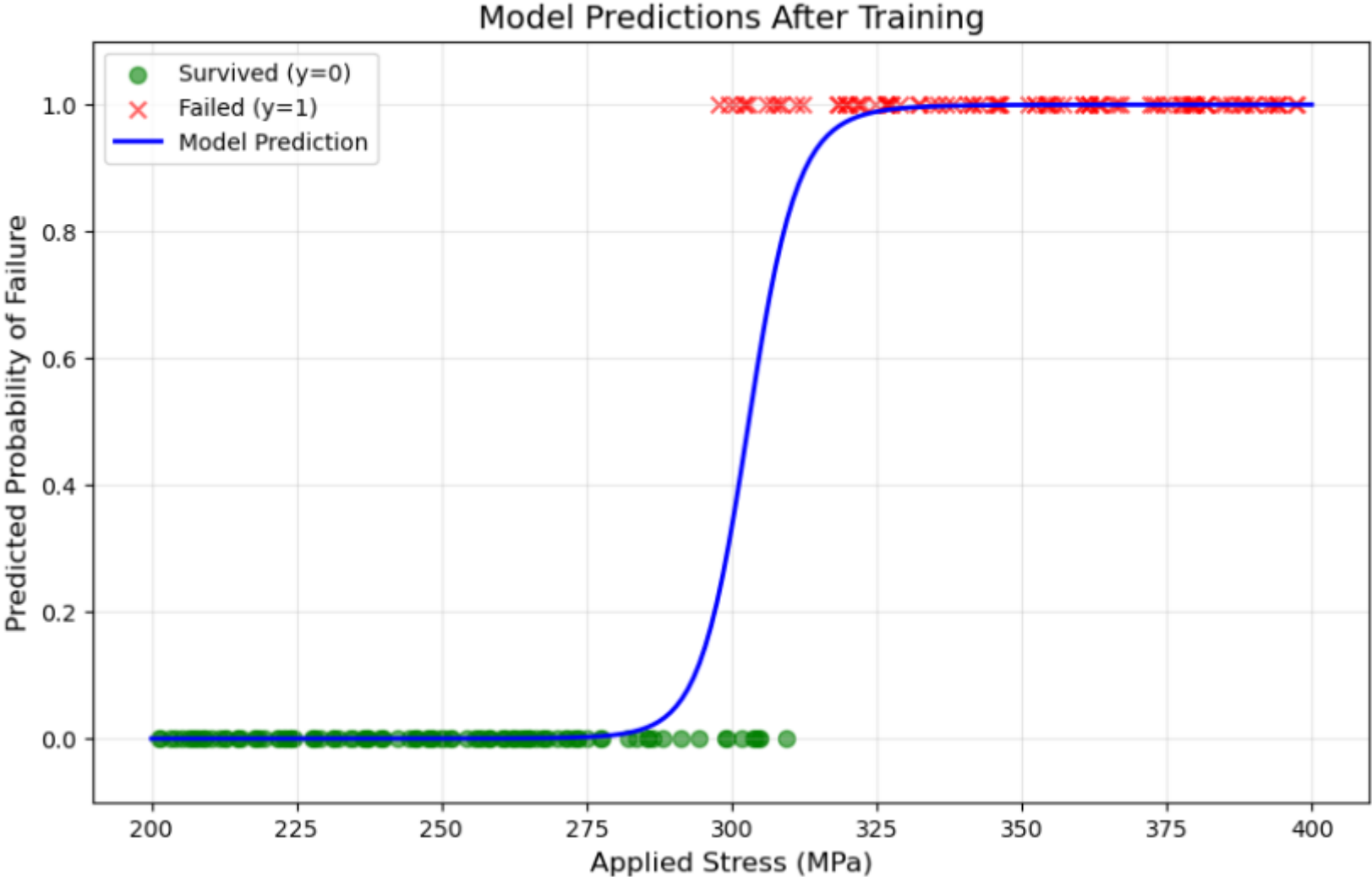
```
for step in range(num_steps):  
    step_loss = 0.0  
  
    # Zero out the gradients  
    for p in model.parameters(): # Grab the list of all trainable parameters in the model  
        if p.grad is not None: # Check if there is already a gradient stored on the parameter  
            p.grad.zero_() # If there is, set it to zero  
  
    # For each point in the minibatch, take a forward pass of the model and add it into step_loss  
    for point in range(batch_size):  
        idx = np.random.randint(len(X))  
        xdata = X_normalized[idx] # get data using the **normalized** input  
        ydata = y[idx]  
        y_pred = model.forward(xdata)  
        loss = binary_cross_entropy(y_pred, ydata)  
        step_loss += loss  
  
    # Take gradient over whole minibatch with autograd  
    step_loss.backward()  
  
    # Take a gradient descent step  
    with torch.no_grad():  
        for p in model.parameters():  
            p -= learning_rate * p.grad
```

Step 1. Use model to evaluate loss

Step 2. Take gradient of loss with respect to model parameters

Step 3. Use gradient to update parameters

Exercise one: probability of failure



A demo of real life complexity that you have the skills to do now

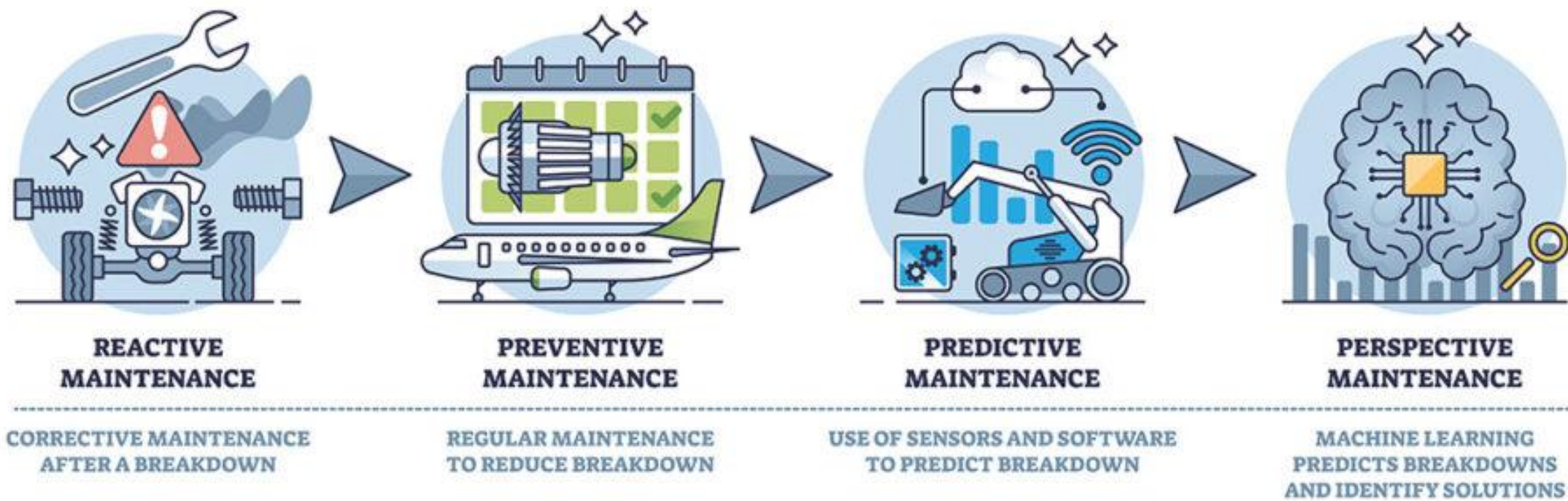
No exercise on this today

On Monday we'll use this to step through final HW together

Many want to machine learn a sensor for final project – this
code will get you 90% of the way there

Exercise two: Predictive maintenance

PREDICTIVE MAINTENANCE



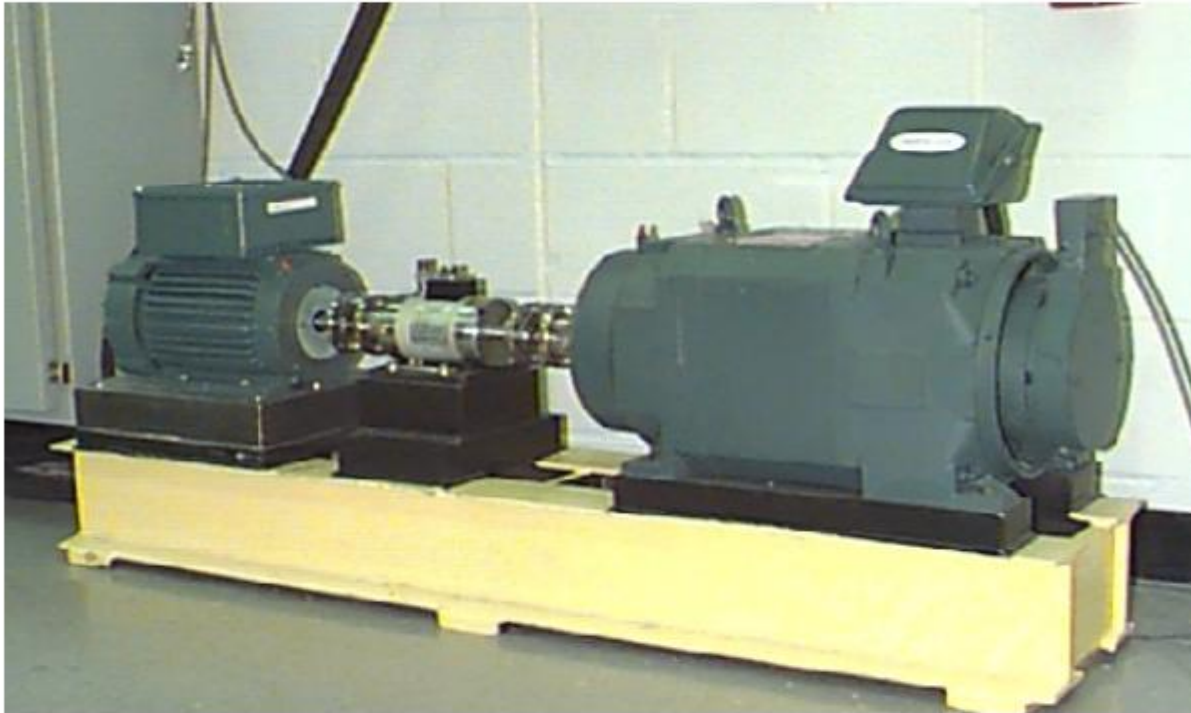
Building a dataset – examples of damaged and undamaged behavior

A ball bearing is a component used to minimize the friction between a rotating shaft, like the drivetrain on a car motor, and the supports that hold it up (e.g. where the driveshaft meets the wheels). The inner cylinder rotates freely within the outer cylinder, rolling on the spherical balls to avoid friction.



Experiment: generate data describing how mechanical system evolves with or without damage

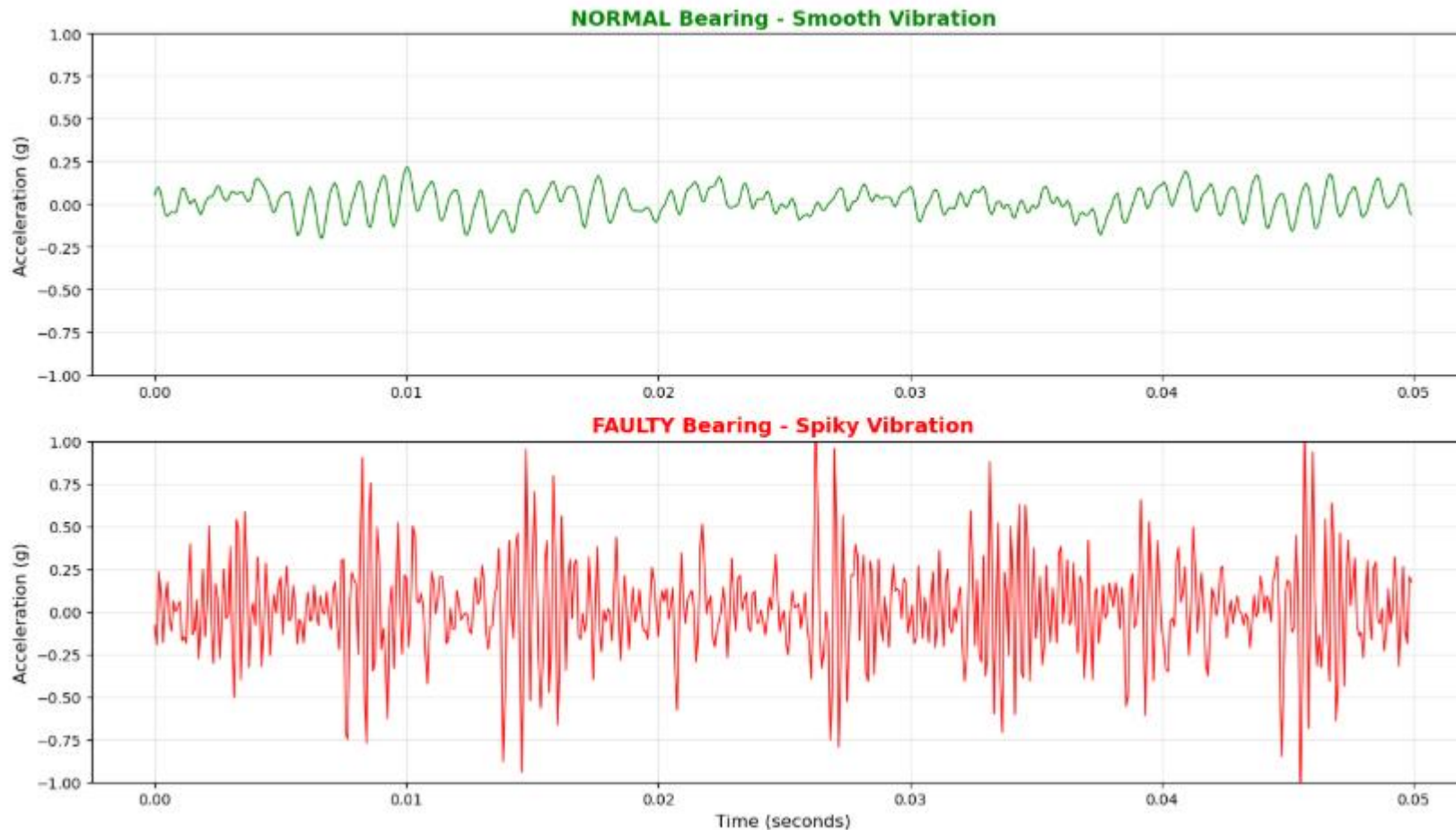
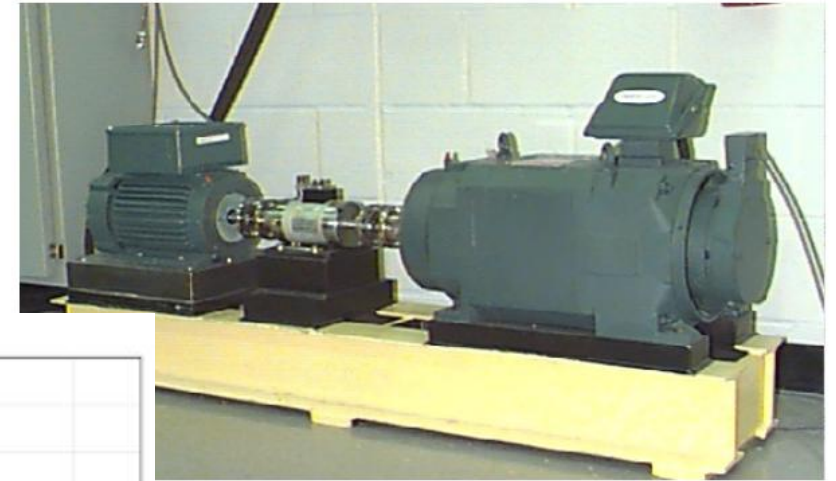
In this experiment, the scientists at Case Western used a motor to drive a ball bearing under load, taking accelerometer measurements off of the bearing to characterize forces as a function of time.



**Samples at 12000 Hz –
that's a lot of data!**

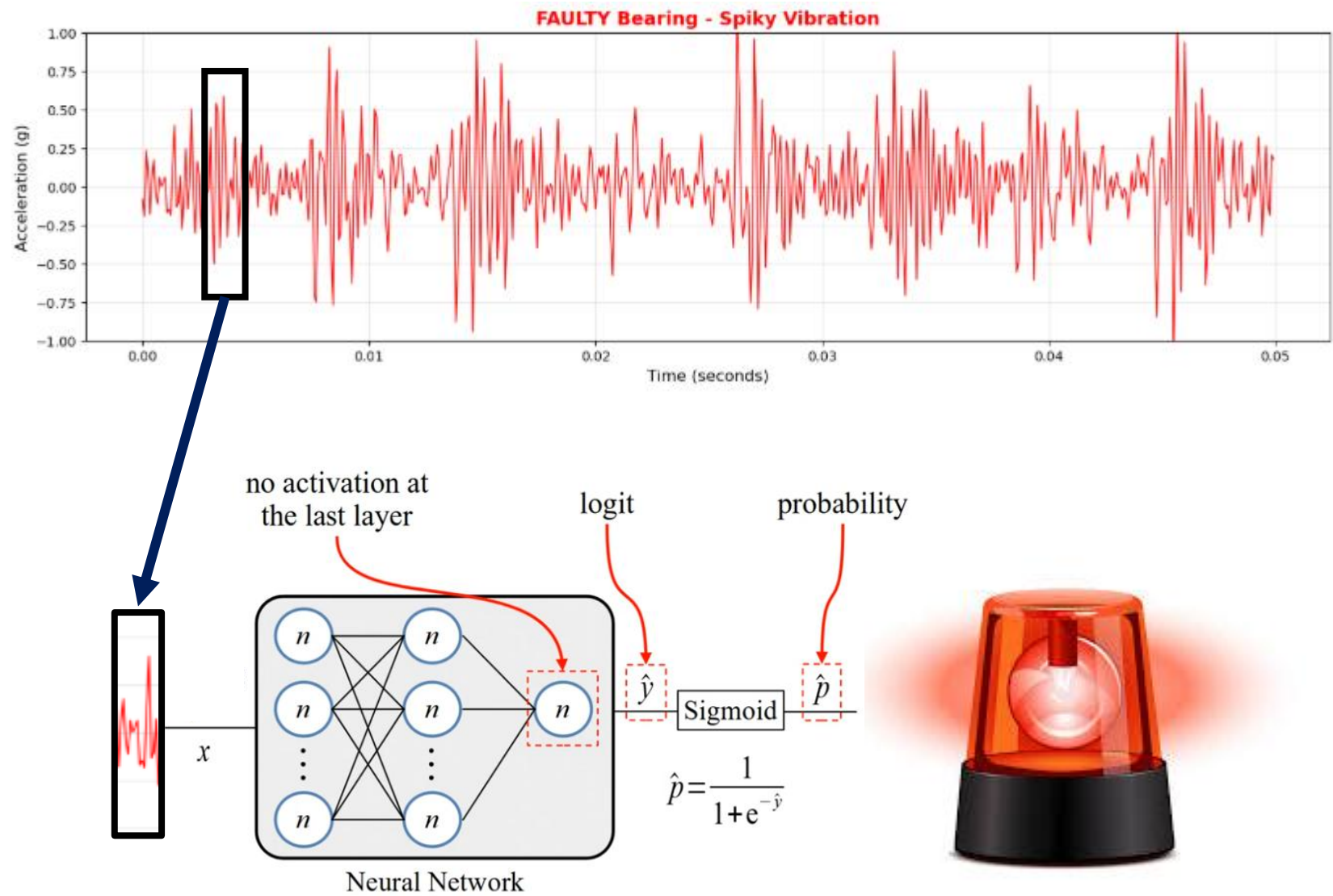
They did this for two setups: in one, they had a perfectly functioning ball bearing. In the other, they intentionally introduced a defect by damaging the bearing.

Always plot your data: what are we working with?



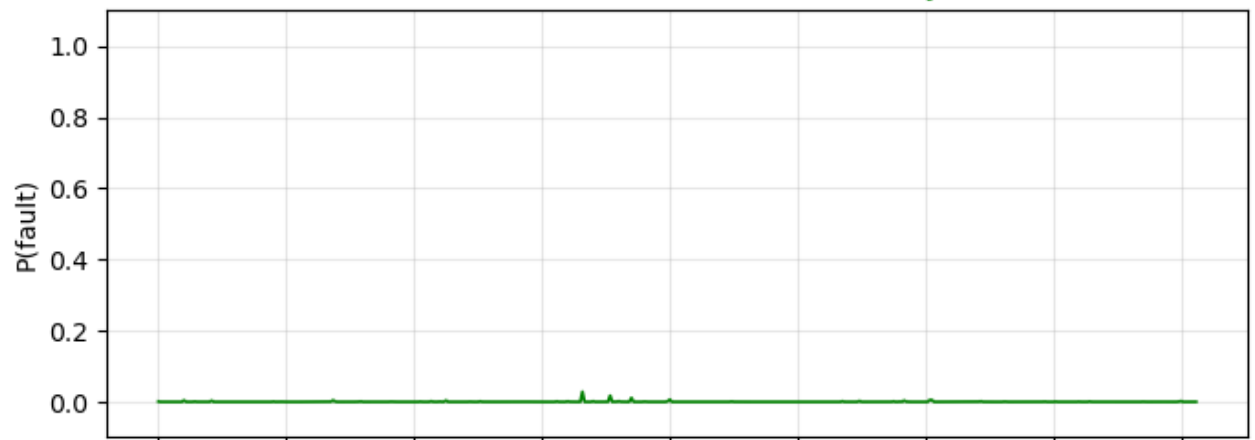
**Taking a small snapshot,
we can ask – is it possible
to detect failure from this
signal?**

Gameplan – feed small window of signal to map onto a true/false failure warning

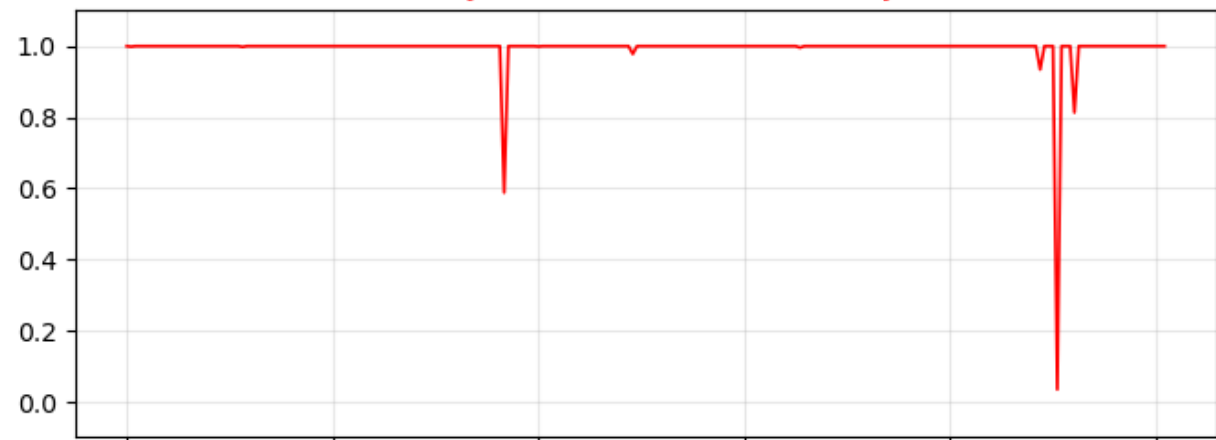


Output – a machine learned failure predictor

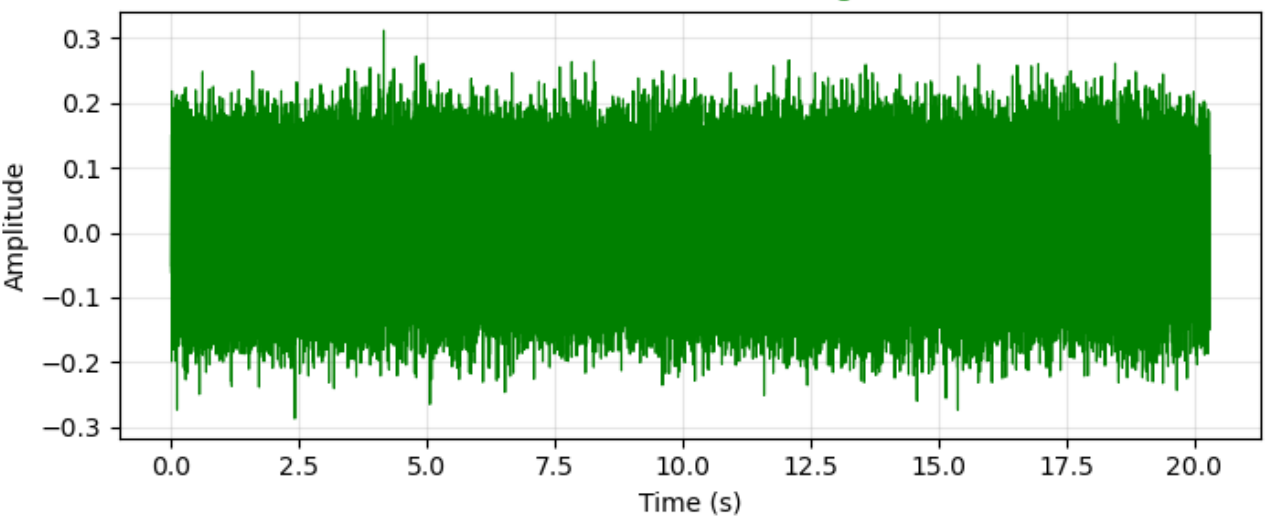
Normal: Predicted Fault Probability



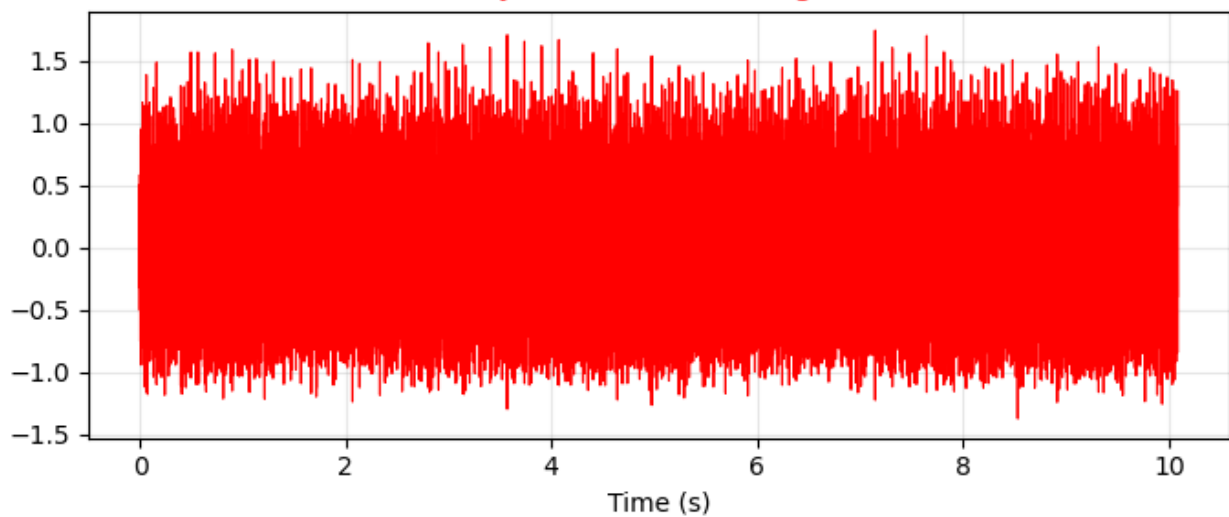
Faulty: Predicted Fault Probability



Normal: Raw Vibration Segment

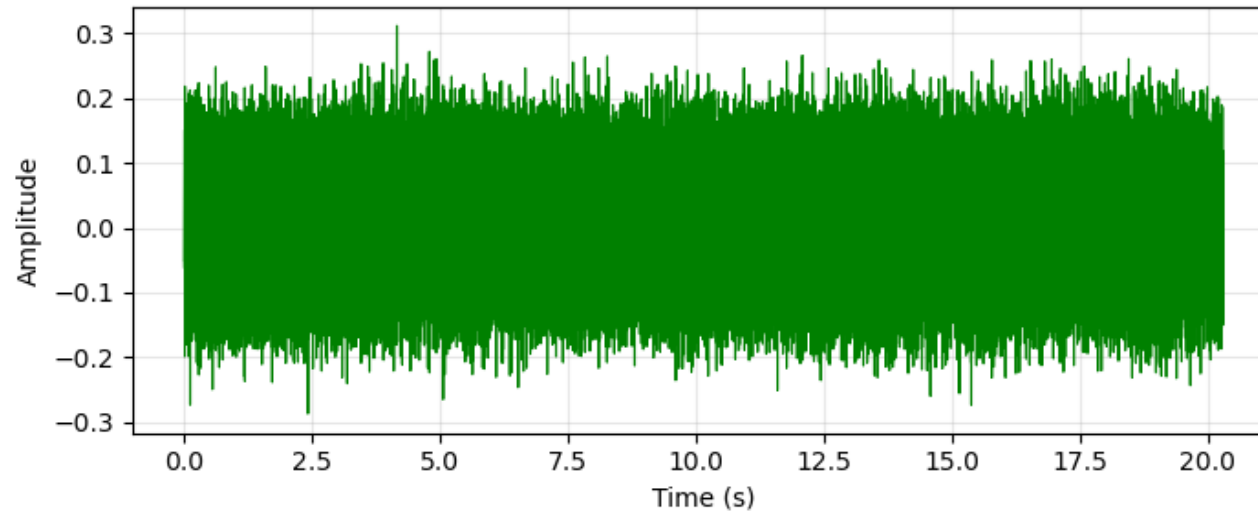


Faulty: Raw Vibration Segment



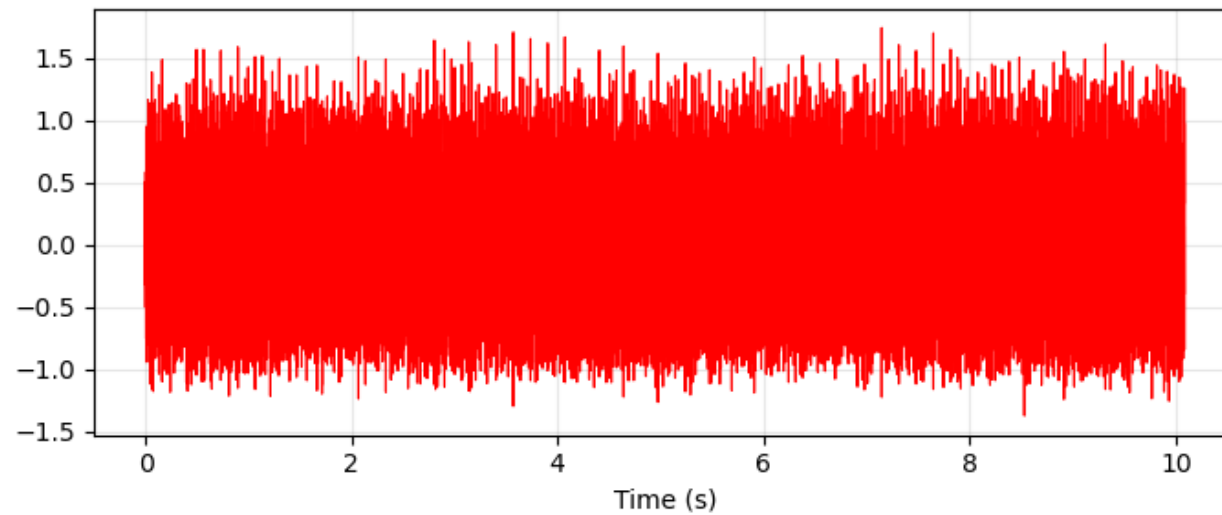
Closing thought: Do we need machine learning for this?

Normal: Raw Vibration Segment



if amplitude > 0.3:
isDamaged = True

Faulty: Raw Vibration Segment

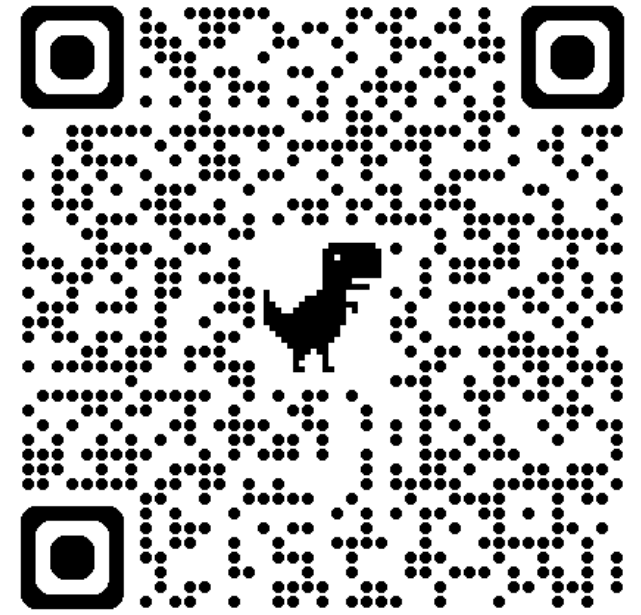


ENGR1050 Final Project Part Ordering F

If you intend to do a Pico based project, please fill out this form to order your parts. Orders must be received by the end of day Wednesday to receive on time.

- The budget per group is \$25/student
- Receiving parts on time will require providing a vendor with a fast delivery time. Digikey and Newark are preferred vendors.
- Before ordering a part, check the estimated delivery time is prior to Dec 3rd.
- Check with Dr. Trask (on Ed is fine) if you are getting something esoteric outside of what is available in class.
- You are responsible for doing research ahead of time to ensure you have appropriate skills to use the part (e.g. no complicated wiring diagrams, no soldering, no 3D printing etc)
- Orders that don't meet requirements will be rejected

To keep things simple, use low tech solutions where possible (e.g. rather than 3D printing a mount for buttons, you can attach to a piece of cardboard). This will keep components secure (wiring and programming) and save you time.



Orders are coming in!

Keep an eye out for emails from me and Peter Bruno – you can pick them up from his office just outside the GM lab